GREENN SCHOOL · MÓDULO LARAVEL

L2

PHP: classes, objetos e organização do código

Apostila do aluno · duração da aula: 2h30

O que você vai aprender

1. Classes, objetos e métodos
2. Construtor, private e herança
3. static, o `::` e namespaces
4. A estrutura de pastas do Laravel
5. O caminho de um pedido: rota → controller → view

Antes de começar

Na L1 você aprendeu a linguagem PHP. Falta uma peça para conseguir **ler** um projeto Laravel: **classes**. Quase todo arquivo do Laravel é uma classe (o controller, o model, a rota chama um método de uma classe), então esta aula começa por elas. Depois você vê como o Laravel organiza essas classes em pastas e o caminho que um pedido faz, do navegador até a página pronta. No site **site-alunos-nu.vercel.app** (módulo Laravel) tem um **mini-Laravel** rodando no navegador para você testar tudo isso.

Prof. Walter Caciano · Greenn School

1 Classes, objetos e métodos

Na L1, um aluno era um array associativo. Quando você tem muitas coisas com as mesmas informações e as mesmas ações, cria uma **classe**: o **molde**. Cada coisa de verdade feita com esse molde é um **objeto**, criado com `new`. A classe é a forma de bolo; os objetos são os bolos.

```php
<?php
class Pedido {
  public array $itens = [];                  // propriedade: um dado do objeto

  public function adicionar(string $item, float $preco): void {   // método: uma ação
    $this->itens[] = ["item" => $item, "preco" => $preco];
  }

  public function total(): float {
    return array_sum(array_column($this->itens, "preco"));
  }
}

$pedido = new Pedido();                      // new cria um objeto
$pedido->adicionar("Pizza", 45);             // -> chama o método
$pedido->adicionar("Refrigerante", 8.5);
echo count($pedido->itens);                  // 2   (-> também acessa a propriedade)
echo $pedido->total();                       // 53.5
```

`$this` quer dizer "**este** objeto aqui": dentro do método, `$this->itens` são os itens do pedido que chamou o método. Cada objeto tem os seus próprios valores: dois pedidos, duas listas de itens.

JavaScript	PHP	Observação
`class Aluno { }`	`class Aluno { }`	igual; o nome começa com maiúscula
`new Aluno("Ana")`	`new Aluno("Ana")`	igual
`aluno.nome`	`$aluno->nome`	a setinha `->` no lugar do ponto (o ponto junta textos)
`aluno.apresentar()`	`$aluno->apresentar()`	método = função dentro da classe
`this.nome`	`$this->nome`	com `$` no `this`, sem `$` no `nome`
`constructor(nome) { }`	`__construct($nome) { }`	dois sublinhados no começo
`class B extends A`	`class B extends A`	igual; `super` vira `parent::`
`Aluno.contar()` (static)	`Aluno::contar()`	coisa da classe usa `::`
`import { Aluno } from ...`	`use App\Models\Aluno;`	importa pelo namespace

2 Construtor, private e herança

O **construtor** `__construct` roda sozinho no `new` e deixa o objeto pronto. No PHP 8 existe a **forma curta**, que o Laravel usa muito: `public` no parâmetro já cria e preenche a propriedade.

```php
<?php
class ContaBancaria {
  private float $saldo = 0;                    // private: só a própria classe mexe

  public function __construct(public string $titular) {}   // forma curta

  public function depositar(float $valor): void {
    if ($valor <= 0) return;                   // a regra fica num lugar só
    $this->saldo += $valor;
  }

  public function verSaldo(): float {
    return $this->saldo;
  }
}

$conta = new ContaBancaria("Ana");
$conta->depositar(200);
echo $conta->verSaldo();                       // 200
$conta->saldo = 1000000;                       // ERRO: Cannot modify private property
```

`public` pode ser usado de qualquer lugar, `private` só de dentro da classe e `protected` de dentro da classe e das filhas. Esconder os dados e obrigar todo mundo a passar pelos métodos se chama **encapsulamento**.

```php
<?php
class Pessoa {
  public function __construct(public string $nome) {}
  public function apresentar(): string { return "Olá, eu sou $this->nome."; }
}

class Professor extends Pessoa {               // herda tudo de Pessoa
  public function __construct(string $nome, public string $materia) {
    parent::__construct($nome);                // chama o construtor da mãe
  }
  public function apresentar(): string {       // troca o método da mãe
    return parent::apresentar() . " Dou aula de $this->materia.";
  }
}

echo (new Professor("Rita", "Laravel"))->apresentar();
// Olá, eu sou Rita. Dou aula de Laravel.
```

Por que isso importa no Laravel: todo controller começa com `class AlunoController extends Controller` e todo model com `class Aluno extends Model`. É por herança que um model vazio, de três linhas, já sabe buscar, salvar e apagar no banco: os métodos vêm prontos da classe `Model`.

A pegadinha nº 1: dentro da classe, a propriedade é sempre `$this->nome`. Um `$nome` sozinho é outra variável, que só existe dentro do método. E depois da setinha não vai `$`: é `$this->nome`, não `$this->$nome`.

3 static, o :: e namespaces

Algumas coisas pertencem à **classe**, não a um objeto: constantes (`const`) e métodos `static` . Elas são usadas sem `new` , com o **quatro-pontos** `::` . Dentro da própria classe, `self::` aponta para ela mesma.

```php
<?php
class Formata {
  const MOEDA = "R$";

  public static function dinheiro(float $valor): string {
    return self::MOEDA . " " . number_format($valor, 2, ",", ".");
  }
}

echo Formata::dinheiro(1234.5);              // R$ 1.234,50   (sem new)
echo Formata::class;                          // "Formata": o nome da classe como texto
```

> **Lendo código Laravel:** `->` é coisa de **objeto** (`$aluno->nome` , `$aluno->save()`); `::` é coisa da **classe** (`Route::get(...)` , `Aluno::all()` , `AlunoController::class`).

Num projeto com centenas de classes, duas podem ter o mesmo nome. O **namespace** é o "sobrenome" da classe, e o `use` importa uma classe de outro namespace, como o `import` do JavaScript:

app/Http/Controllers/AlunoController.php

```php
<?php
namespace App\Http\Controllers;               // o "sobrenome" desta classe

use App\Models\Aluno;                          // importa a classe Aluno de App\Models

class AlunoController extends Controller {
  public function index() {
    $alunos = Aluno::all();                    // sem o use, o PHP procuraria
App\Http\Controllers\Aluno
  }
}
```

No Laravel, **cada classe fica no seu próprio arquivo**, e o namespace segue a pasta: `App\Models\Aluno` mora em `app/Models/Aluno.php` . Por causa dessa regra, o **Composer** (o gerenciador de pacotes do PHP) sabe sozinho qual arquivo carregar quando você usa uma classe: é o **autoload**. Você nunca escreve `require` .

4 A estrutura de pastas do Laravel

Um projeto novo (`laravel new escola`) já vem organizado, e cada tipo de arquivo tem o seu lugar. Isso se chama **convenção**: como todo projeto Laravel segue a mesma estrutura, você abre o projeto de outra pessoa e sabe onde procurar cada coisa. As pastas com ★ são as que você vai abrir quase todo dia.

```
escola/
app/
  Http/Controllers/     ★ controllers: recebem o pedido e decidem a resposta
  Models/               ★ models: uma classe por tabela do banco
bootstrap/                o "liga" do framework (quase nunca se mexe)
config/                   configurações: nome do app, banco, e-mail…
database/
  migrations/             a estrutura das tabelas (aula L5)
public/                   a ÚNICA pasta aberta ao navegador: index.php, imagens
resources/
  views/                ★ views: o HTML das páginas (Blade)
  css/  js/               o seu CSS e JavaScript
routes/
  web.php               ★ rotas: qual URL chama qual controller
storage/                  arquivos enviados, cache e logs
tests/                    testes automáticos
vendor/                   o código do Laravel e dos pacotes (não mexa)
.env                      senhas e dados do ambiente (NUNCA vai para o Git)
artisan                   o comando do terminal: php artisan …
composer.json             a lista de pacotes do projeto (o package.json do PHP)
```

Você vai criar…	Fica em	Comando que cria
uma rota	`routes/web.php`	(você escreve no arquivo)
um controller	`app/Http/Controllers/AlunoController.php`	`php artisan make:controller AlunoController`
um model	`app/Models/Aluno.php`	`php artisan make:model Aluno`
uma view	`resources/views/alunos/index.blade.php`	`php artisan make:view alunos.index`
uma tabela	`database/migrations/…_create_alunos_table.php`	`php artisan make:migration create_alunos_table`

Por que só a pasta public? O navegador só enxerga o que está em `public/`. O resto do projeto (o código, o `.env` com as senhas) fica fora do alcance de quem visita o site. Todo pedido entra pelo `public/index.php`, que liga o Laravel e entrega o pedido para as rotas.

5 O caminho de um pedido: rota → controller → view

Este é o caminho de **todo** pedido num projeto Laravel. Quando alguém abre `escola.com/alunos`:



⑥ o HTML pronto volta para o navegador, que mostra a página

② routes/web.php

```php
<?php
use App\Http\Controllers\AlunoController;
use Illuminate\Support\Facades\Route;

Route::get('/alunos', [AlunoController::class, 'index']);       // /alunos → método index
Route::get('/alunos/{id}', [AlunoController::class, 'show']);   // {id} chega como parâmetro
```

③ app/Http/Controllers/AlunoController.php

```php
<?php
namespace App\Http\Controllers;

use App\Models\Aluno;

class AlunoController extends Controller
{
    public function index()
    {
        $alunos = Aluno::all();                              // ④ pede os dados ao model
        return view('alunos.index', ['alunos' => $alunos]);  // ⑤ entrega para a view
    }

    public function show(string $id)                         // o {id} da rota chega aqui
    {
        return view('alunos.show', ['aluno' => Aluno::findOrFail($id)]);
    }
}
```

⑤ resources/views/alunos/index.blade.php

```blade
<h1>Alunos da turma</h1>
<ul>
  @foreach ($alunos as $aluno)
    <li>{{ $aluno->nome }}</li>
  @endforeach
</ul>
```

`'alunos.index'` quer dizer o arquivo `resources/views/alunos/index.blade.php` (o ponto separa as pastas). Cada chave do array vira uma variável dentro da view: `'alunos' => $alunos` vira `$alunos`. O Blade (`{{ }}`, `@foreach`) é o PHP gerando HTML que você viu no fim da L1, com uma sintaxe mais limpa; ele é o assunto da L3.

Cada um com a sua tarefa: a rota não busca dados, o controller não escreve HTML e a view não fala com o banco. Quando der um erro, essa divisão diz onde procurar: URL errada ou 404, olhe a rota; dado errado, olhe o controller e o model; página feia, olhe a view.

E o 404? Se nenhuma rota bate com a URL pedida, o Laravel responde **404 | Not Found**. Antes de procurar o erro no código, confira se a rota existe, com o comando `php artisan route:list`. No playground, o mini-Laravel (`Route::get`, `view()`, `visitar()`) faz esse mesmo caminho numa versão de brinquedo, para você testar sem instalar nada.

6 Exercícios

Do 1 ao 3, classes, métodos e construtor; 4 a 7, private, herança, listas de objetos e static; 8 a 10, a estrutura do Laravel e o caminho do pedido. Em cada um, compare a sua saída com a esperada: cada letra, espaço e acento conta.

01 Primeira classe

NÍVEL 1

Classe `Livro` com título e preço; um objeto criado com `new`.

Saída esperada: Dom Casmurro custa R$ 39,90

02 Métodos do retângulo

NÍVEL 1

Métodos `area()` e `perimetro()` usando `$this`.

Saída esperada: Área: 12 / Perímetro: 14

03 Construtor

NÍVEL 1

`__construct` com nome e curso, e o método `apresentar()`.

Saída esperada: Ana estuda Laravel / Caio estuda JavaScript

04 Conta protegida

NÍVEL 2

Saldo `private` e os métodos depositar, sacar e verSaldo.

Saída esperada: Saldo: 150 / Saldo insuficiente / Saldo: 150

05 Herança

NÍVEL 2

`Cachorro` e `Gato` estendem `Animal` e trocam o método `som()`.

Saída esperada: Rex diz: Au au! / Mimi diz: Miau!

06 Lista de produtos

NÍVEL 2

`foreach` numa lista de objetos, somando os preços.

Saída esperada: - Caneca: R$ 25,00 ... Total: R$ 129,90

07 Método estático

NÍVEL 2

`Formata::dinheiro()`: um método `static`, chamado sem `new`.

Saída esperada: R$ 25,00 / R$ 1.234,50

08 Onde fica cada arquivo

NÍVEL 2

Uma função com `match` que diz a pasta do Laravel de cada arquivo.

Saída esperada: AlunoController.php → app/Http/Controllers ...

09 Rota e controller

NÍVEL 2

`PaginaController` com `home()` e `sobre()`, e as rotas para eles.

Saída esperada: Página inicial / Sobre a Greenn School / 404 | Página não encontrada

10 O caminho completo

DESAFIO

Rota `/produtos` → `ProdutoController` → view com um `<li>` por produto.

Saída esperada: HTML renderizado na prévia

Travou? Volte ao exemplo da seção correspondente, rode o código no site e mude uma coisa por vez para entender o que cada parte faz. Um `dump($objeto)` mostra tudo o que tem dentro de um objeto.